# COMPONENTES BÁSICOS

EJEMPLO 1: Text, View, Image, StatusBar, estilos en hoja de estilos.

Archivo App.js

```
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import {Text, View, Image} from 'react-native';
import styles from './styles/styles.js';

export default function App() {
 /*Esto es un comentario en JS*/
 const URL =
"https://imgs.search.brave.com/tfqaxLA5zQy59PzCLhtC-tNAmBqPnyPlwnpqN9jyihU/rs:fit:500:0:0:0/g:ce/aHR0cHM6Ly90MS5l/YS5sdG1jZG4uY29t/L2VzL3Bvc3RzLzgv/OS85L3RpZW5lX21h/bF9hbGllbnRvXzIy/OTk4XzdfNjAwLmpw/Zw";
 const name = "Pilar González";

 return (
   <View style={styles.container}>
    {/*Esto es un comentario en JSX */}

    <Text style={styles.title}>Hola, {name}</Text>

    <Image source={{uri:URL}} style={{width: 200, height: 200}}/>
    <StatusBar style="auto" />
   </View>
 );
}
```

Archivo styles/styles.js

```
import { StyleSheet } from "react-native";

export default StyleSheet.create({
   container: {
      flex: 1,
      backgroundColor: '#9a6d6dff',
      alignItems: 'center',
      justifyContent: 'center',
   },
   title: {
      fontSize: 30,
      color:'#7ac13cff',
   },

});
```

# HOOK useState

EJEMPLO 1: TextInput más useState

miApp > components > JS TextInput_ejemplo.js > ...

```
1  import React, { useState } from 'react';
2  import { TextInput, Text, View } from 'react-native';
3
4  export default function Formulario() {
5    const [name, setName] = useState('');
6
7    return (
8      <View style={{ padding: 20 }}>
9        <TextInput
10         style={{
11           height: 40,
12           borderColor: 'gray',
13           borderWidth: 1,
14           paddingHorizontal: 10,
15         }}
16         placeholder="Escribe tu nombre"
17         value={name}
18         onChangeText={texto => setName(texto)}
19       />
20       <Text>Hola, {name}</Text>
21     </View>
22   );
23 }
```

miApp > JS App.js > ...

```
1  import React from 'react';
2  import { StatusBar } from 'expo-status-bar';
3  import {Text, View, Image} from 'react-native';
4  import styles from './styles/styles.js';
5  import Formulario from './components/TextInput_ejemplo.js';
6
7  export default function App() {
8    return (
9      <View style={styles.container}>
10       <Formulario />
11       <StatusBar style="auto" />
12     </View>
13   );
14 }
```

EJEMPLO 2: Contador

miApp > components > JS Ejemplo1_contador.js > ...

```jsx
import React, { useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function Contador() {
  const [contador, setContador] = useState(0);  // Estado inicial: 0

  const incrementar = () => {
    setContador(contador + 1);  // Actualiza el estado
  };

  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Contador: {contador}</Text>
      <TouchableOpacity style={styles.boton} onPress={incrementar}>
        <Text style={styles.textoBoton}>+1</Text>
      </TouchableOpacity>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  titulo: { fontSize: 24, marginBottom: 20 },
  boton: { backgroundColor: 'blue', padding: 10, borderRadius: 5 },
  textoBoton: { color: 'white', fontSize: 18 },
});
```

miApp > JS App.js > ...

```jsx
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import {Text, View, Image} from 'react-native';
import styles from './styles/styles.js';
import Contador from './components/Ejemplo1_contador.js';

export default function App() {
  return (
    <View style={styles.container}>
      <Contador />
      <StatusBar style="auto" />
    </View>
  );
}
```

EJEMPLO 3: dos objetos y operador spread …
Maneja dos objetos de estado independientes: uno para datos de una persona (***persona***) y otro para datos de una ubicación (***ubicacion***). Cada uno se actualiza  en tiempo real por separado mediante botones.
El **operador …** (spread) toma los elementos de un **array** u **objeto** y los "copia" o "expande" en un nuevo lugar. No modifica el original. Sintaxis: Tres puntos seguidos (...) antes del nombre del array/objeto.
Más info de spread: https://www.w3schools.com/react/react_es6_spread.asp

miApp > components > JS DosEstados.js > DosEstados

```jsx
import React, { useState } from 'react';
import { View, Text, TouchableOpacity } from 'react-native';
import styles from '../styles/styles.js';

export default function DosEstados (){
   const [persona, setPersona] = useState({ name: 'Juan', age: 25 });
  const [ubicacion, setUbicacion] = useState({ city: 'Madrid', country: 'España' });

  const actualizarPersona = () => {
    setPersona({ ...persona, age: persona.age + 1 });
  };

  const actualizarUbicacion = () => {
    setUbicacion({ ...ubicacion, city: 'Barcelona' });
  };

  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Persona: {persona.name}, Edad: {persona.age}</Text>
      <TouchableOpacity style={styles.boton} onPress={actualizarPersona}>
        <Text style={styles.textoBoton}>Cumplir años (+1 edad)</Text>
      </TouchableOpacity>

      <Text style={styles.titulo}>Ubicación: {ubicacion.city}, {ubicacion.country}</Text>
      <TouchableOpacity style={styles.boton} onPress={actualizarUbicacion}>
        <Text style={styles.textoBoton}>Cambiar ciudad</Text>
      </TouchableOpacity>
    </View>
  );
}
```

SOLUCIÓN AL RETO: CONTADOR BIDIRECCIONAL

```jsx
import React, { useState } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function Contador2() {
  const [contador, setContador] = useState(5);

  const incrementar = () => {
    setContador(contador + 1);
  };

  const decrementar = () => {
    setContador(contador > 0 ? contador - 1 : 0);
  };

  return (
    <View style={styles.container}>
      <Text
        style={[
          styles.titulo,
          contador === 0 && { color: 'red' }
        ]}
      >
        {contador === 0 ? '¡No más!' : `Contador: ${contador}`}
      </Text>

      <TouchableOpacity style={styles.boton} onPress={incrementar}>
        <Text style={styles.textoBoton}>+1</Text>
      </TouchableOpacity>
```

Continúa el código…

```jsx
      <TouchableOpacity style={[styles.boton, styles.botonMenos]} onPress={decrementar}>
        <Text style={styles.textoBoton}>-1</Text>
      </TouchableOpacity>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center'
  },
  titulo: {
    fontSize: 24,
    marginBottom: 20,
    color: 'black'
  },
  boton: {
    backgroundColor: 'blue',
    padding: 10,
    borderRadius: 5,
    marginBottom: 10
  },
  botonMenos: {
    backgroundColor: 'red'
  },
  textoBoton: {
    color: 'white',
    fontSize: 18,
    textAlign: 'center'
  },
});
```

# React navigation

Para los siguientes ejemplos crea un nuevo proyecto Expo llamado ***navigationApp*** con:
***npx create-expo-app navigationApp --template blank***

Instala la librería react-navigation y sus dependencias:
***npm install @react-navigation/native***

***npx expo install react-native-screens react-native-safe-area-context***

Cada tipo de navegación necesita instalar su librería concreta. Instala todos los tipos.

***npm install @react-navigation/native-stack***
***npm install @react-navigation/bottom-tabs***
***npm install @react-navigation/drawer***

Crea siguiente estructura en tu proyecto:
/ carpeta
Proyecto: se llama navigationApp

/navigationApp
package.json
App.js
/components
/screens
|---- HomeScreen.js
|---- DetailsScreen.js
|---- ProfileScreen.js
|---- SettingsScreen.js

/navigation
|---- StackNavigator.js
|---- TabNavigator.js
|---- DrawerNavigator.js
/styles
|---- styles.js

Para todos los ejemplos usaremos el mismo código en **App.js.**

navigationApp > App.js > App

```jsx
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import {Text, View, Image} from 'react-native';
import styles from './styles/styles.js';

import { NavigationContainer } from '@react-navigation/native';
import StackNavigator from './navigation/StackNavigator.js';

export default function App() {
  return (
    <NavigationContainer>
      <View style={styles.container}>
          <Text>Inicio de navegación</Text>

          <StackNavigator />
      </View>

      <StatusBar style="auto" />
    </NavigationContainer>
  );
}
```

# PRIMER TIPO DE NAVEGACIÓN: STACK (PILA)

## StackNavigator.js

StackNavigator.js U ● | styles.js U | HomeScreen.js U | DetailsScreen.js U | ProfileScreen.js U | JS

navigationApp > navigation > StackNavigator.js > ...

```jsx
import React from 'react';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from '../screens/HomeScreen.js';
import ProfileScreen from '../screens/ProfileScreen.js';
import DetailsScreen from '../screens/DetailsScreen.js';
import styles from '../styles/styles.js';

const Stack = createNativeStackNavigator();

export default function StackNavigator() {
  return (
    <Stack.Navigator
        initialRouteName='Home'
        screenOptions={{
            headerStyle: { backgroundColor: '#f4511e' },
            headerTintColor: '#fff',
            headerTitleStyle: { fontWeight: 'bold' },
        }}
    >
        <Stack.Screen name="Home" component={HomeScreen} options={{title:'Inicio'}}/>
        <Stack.Screen name="Detalles" component={DetailsScreen} options={{title:'Detalles'}}/>
        <Stack.Screen name="Perfil" component={ProfileScreen} options={{title:'Perfil'}}/>

    </Stack.Navigator>
  );
}
```

## HomeScreen.js

JS StackNavigator.js U ● JS styles.js U JS HomeScreen.js U × JS DetailsScreen.js U

navigationApp > screens > JS HomeScreen.js > HomeScreen

```
1   import React from 'react';
2   import { View, Text, Pressable } from 'react-native';
3
4   import styles from '../styles/styles.js';
5
6   export default function HomeScreen({navigation}) {
7     return (
8       <View style={styles.container}>
9           <Text>Pantalla Home</Text>
10
11          <Pressable
12              style={styles.boton}
13              onPress={()=>navigation.navigate("Detalles")}
14          >
15              <Text style={styles.textoBoton}>Ir a detalles</Text>
16          </Pressable>
17      </View>
18    );
19  }
```

**DetailsScreen.js**

navigationApp > screens > JS DetailsScreen.js > DetailsScreen

```
1   import React from 'react';
2   import { View, Text, Pressable } from 'react-native';
3
4   import styles from '../styles/styles.js';
5
6   export default function DetailsScreen({navigation}) {
7     return (
8       <View style={styles.container}>
9           <Text>Pantalla de Detalles</Text>
10
11          <Pressable
12              style={styles.boton}
13              onPress={()=>navigation.navigate("Perfil")}
14          >
15              <Text style={styles.textoBoton}>Ir al Perfil</Text>
16          </Pressable>
17
18          <Pressable
19              style={styles.boton}
20              onPress={()=>navigation.goBack()}
21          >
22              <Text style={styles.textoBoton}>Volver</Text>
23          </Pressable>
24      </View>
25    );
26  }
```

**ProfileScreen.js**

navigationApp > screens > JS ProfileScreen.js > ...

```jsx
import React from 'react';
import { View, Text, Pressable } from 'react-native';

import styles from '../styles/styles.js';

export default function ProfileScreen( ) {
  return (
    <View style={styles.container}>
        <Text>Pantalla de Perfil</Text>
    </View>
  );
}
```

**SettingsScreen.js**

navigationApp > screens > JS SettingsScreen.js > ...

```jsx
import React from 'react';
import { View, Text, Pressable } from 'react-native';

import styles from '../styles/styles.js';

export default function SettingsScreen( ) {
  return (
    <View style={styles.container}>
        <Text>Pantalla de Configuración</Text>
    </View>
  );
}

```

En App.js se llama al componente ***StackNavigator***. Todo dentro del contenedor de navegación principal, el componente ***NavigationContainer***. Fíjate que para utilizar los métodos ***navigate*** y ***goBack*** se utiliza el componente de React Native ***navigation*** que se le pasa como props a las pantallas que usan esos métodos.

## SEGUNDO TIPO DE NAVEGACIÓN: TABS (PESTAÑAS)

**TabNavigator.js**

navigationApp > navigation > JS TabNavigator.js > TabNavigator

```
1   import React from 'react';
2   import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
3   import HomeScreen from '../screens/HomeScreen';
4   import DetailsScreen from '../screens/DetailsScreen';
5   import ProfileScreen from '../screens/ProfileScreen';
6
7   const Tab = createBottomTabNavigator();
8
9   export default function TabNavigator() {
10    return (
11      <Tab.Navigator
12        initialRouteName="Home"
13        screenOptions={{
14          tabBarActiveTintColor: '#007AFF',
15          tabBarInactiveTintColor: 'gray',
16          headerShown: false,
17        }}
18      >
19        <Tab.Screen
20          name="Home"
21          component={HomeScreen}
22          options={{
23            tabBarLabel: 'Inicio',
24          }}
25        />
26        <Tab.Screen
27          name="Details"
28          component={DetailsScreen}
29          options={{
30            tabBarLabel: 'Detalles',
31          }}
32        />
33        <Tab.Screen
34          name="Profile"
35          component={ProfileScreen}
36          options={{
37            tabBarLabel: 'Perfil',
38          }}
39        />
40      </Tab.Navigator>
41    );
42  }
```

Ahora las pantallas las hemos definido en TabNavigator y aparecerán **pestañas en la parte inferior** para navegar entre pantallas. Se sitúan en la parte inferior porque estamos utilizando la librería bottom-tabs.

Modificamos **App.js** para que llame a TabNavigator, haciendo el import de ***TabNavigator*** necesario.

## NESTED NAVIGATION (NAVEGACIÓN MIXTA): Stack principal y tabs anidado

En ocasiones es necesario tener dos o más tipos de navegación, sin embargo hay que tener en cuenta a la hora de navegar cuál es la navegación a la que pertenece la pantalla concreta.
Usaremos el método ***getParent( )*** para referirnos a la navegación padre cuando sea necesario. Lo vemos en el código.
En este ejemplo tendremos un navegador principal tipo pila, definido en *MainStackNavigator* que llama a una pantalla con navegación tipo Tabs, *TabNavigator*.

Crea un nuevo archivo **MainStackNavigator.js** dentro de la carpeta **/navigation** con este contenido:

navigationApp > navigation > JS MainStackNavigator.js > ...

```
1   import React from 'react';
2   import { createNativeStackNavigator } from '@react-navigation/native-stack';
3   import TabNavigator from './TabNavigator';
4   import DetailsScreen from '../screens/DetailsScreen';
5   import ProfileScreen from '../screens/ProfileScreen';
6
7   const Stack = createNativeStackNavigator();
8
9   export default function MainStackNavigator() {
10    return (
11      <Stack.Navigator
12        initialRouteName="Tabs"
13        screenOptions={{
14          headerStyle: { backgroundColor: '#f4511e' },
15          headerTintColor: '#fff',
16          headerTitleStyle: { fontWeight: 'bold' },
17        }}
18      >
19        <Stack.Screen
20          name="Tabs"
21          component={TabNavigator}
22          options={{ headerShown: false }}
23        />
24        <Stack.Screen name="Detalles" component={DetailsScreen} />
25        <Stack.Screen name="Perfil" component={ProfileScreen} />
26      </Stack.Navigator>
27    );
28  }
29
```

Fíjate que  ahora en lugar de definir la pantalla Home, **se define Tabs que será la pantalla que contenga la navegación por pestañas**. El resto de pantallas también se definen aquí, en este caso **Detalles** y **Perfil**.

**Este será el componente al que llamarás en App.js.**

navigationApp > App.js > ...

```
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import {Text, View, Image} from 'react-native';
import styles from './styles/styles.js';

import { NavigationContainer } from '@react-navigation/native';
import MainStackNavigator from './navigation/MainStackNavigator.js';

export default function App() {
  return (
    <NavigationContainer>
          <MainStackNavigator/>
     <StatusBar style="auto" />
    </NavigationContainer>
  );
}
```

Si dejamos el resto de archivos tal cual, no se mostrará nada porque la pantalla ***HomeScreen*** ahora tiene que obtener la navegación de su padre (el stack) para poder navegar. Para ello se usa el método de ***navigation getParent()***, que devuelve el objeto de navegación del navigator padre.

Por otro lado, TabNavigator no incluirá ya las pantallas Detalles ni Perfil, ya que se accede a ellas con la navegación tipo stack. Por ello, agregaremos la pantalla SettingsScreen para que al menos tengamos dos pestañas.
Aquí está el código de **TabNavigator.js**.

navigationApp > navigation > TabNavigator.js > TabNavigator

```
import React from 'react';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import HomeScreen from '../screens/HomeScreen';
import SettingsScreen from '../screens/SettingsScreen';

const Tab = createBottomTabNavigator();

export default function TabNavigator() {
  return (
    <Tab.Navigator
      initialRouteName="Home"
      screenOptions={{
        tabBarActiveTintColor: '#007AFF',
        tabBarInactiveTintColor: 'gray',
        headerShown: false,
      }}
    >
      <Tab.Screen
        name="Home"
        component={HomeScreen}
        options={{
          tabBarLabel: 'Inicio',
        }}
      />

      <Tab.Screen
        name="Settings"
        component={SettingsScreen}
        options={{
          tabBarLabel: 'Configuración',
        }}

      />

    </Tab.Navigator>
  );
}
```

Modificamos el código de HomeScreen que al definirla dentro de TabNavigator se ha de usar getParent() para obtener la navegación del Stack.

**HomeScreen.js**.

navigationApp > screens > JS HomeScreen.js > HomeScreen

```
import React from 'react';
import { View, Text, Pressable } from 'react-native';

import styles from '../styles/styles.js';

export default function HomeScreen({navigation}) {
  return (
    <View style={styles.container}>
        <Text>Pantalla Home</Text>

        <Pressable
            style={styles.boton}

            onPress={()=>{
              navigation.getParent().navigate("Detalles")
              console.log("Navegando a Detalles");
            }}
        >
            <Text style={styles.textoBoton}>Ir a detalles</Text>
        </Pressable>

    </View>
  );
}
```

**SettingsScreen.js** también tiene que acceder a la navegación del padre, este es su código:

navigationApp > screens > JS SettingsScreen.js > ...

```
import React from 'react';
import { View, Text, Pressable } from 'react-native';

import styles from '../styles/styles.js';

export default function SettingsScreen( {navigation}) {
  return (
    <View style={styles.container}>
        <Text>Pantalla de Configuración</Text>

          <Pressable
              style={styles.boton}
              onPress={()=>navigation.getParent().navigate("Detalles")}

            >
            <Text style={styles.textoBoton}>Ir a Detalles</Text>
            </Pressable>
    </View>
  );
}
```

DetailsScreen y ProfileScreen no necesitan ser modificados, ya que se encuentran definidos dentro de la navegación principal, el stack. A ProfileScreen le vamos a agregar botones para volver e ir a detalles. Comprueba los códigos de estos componentes que ya tienes y verifica que no han cambiado.

**DetailsScreen.js**

navigationApp > screens > JS DetailsScreen.js > DetailsScreen

```
1   import React from 'react';
2   import { View, Text, Pressable } from 'react-native';
3
4   import styles from '../styles/styles.js';
5
6   export default function DetailsScreen({navigation}) {
7     return (
8       <View style={styles.container}>
9           <Text>Pantalla de Detalles</Text>
10
11          <Pressable
12              style={styles.boton}
13              onPress={()=>navigation.navigate("Perfil")}
14          >
15              <Text style={styles.textoBoton}>Ir al Perfil</Text>
16          </Pressable>
17
18          <Pressable
19              style={styles.boton}
20              onPress={()=>navigation.goBack()}
21          >
22              <Text style={styles.textoBoton}>Volver</Text>
23          </Pressable>
24      </View>
25    );
26  }
```

**ProfileScreen.js**

navigationApp > screens > JS ProfileScreen.js > ...

```
1   import React from 'react';
2   import { View, Text, Pressable } from 'react-native';
3
4   import styles from '../styles/styles.js';
5
6   export default function ProfileScreen({navigation} ) {
7     return (
8       <View style={styles.container}>
9           <Text>Pantalla de Perfil</Text>
10
11          <Pressable
12            style={styles.boton}
13            onPress={()=>navigation.navigate("Tabs")}
14
15          >
16          <Text style={styles.textoBoton}>Ir al Inicio</Text>
17          </Pressable>
18      </View>
19    );
20  }
```

El **problema** de la navegación mixta, **nested navigation**, es que al tener dos componentes navigator en ocasiones React no reconoce a qué pantalla tiene que navegar. Cuando se llama a ***navigate('Screen')***, busca primero en el *navigator* actual (hijo), y **solo si no encuentra *Screen***, **sube al padre**. Pero esto **falla en casos complejos** (nombres duplicados o nesting profundo), por lo que se usa ***getParent()*** para forzar el acceso directo al padre.

# TERCER TIPO DE NAVEGACIÓN: DRAWER (CAJÓN)

El **Drawer Navigator** crea un menú lateral deslizable, común en apps con muchas secciones.

En este ejemplo partimos el proyecto *navigationApp*.
Instala las librerías necesarias en el proyecto si no las has instalado ya:

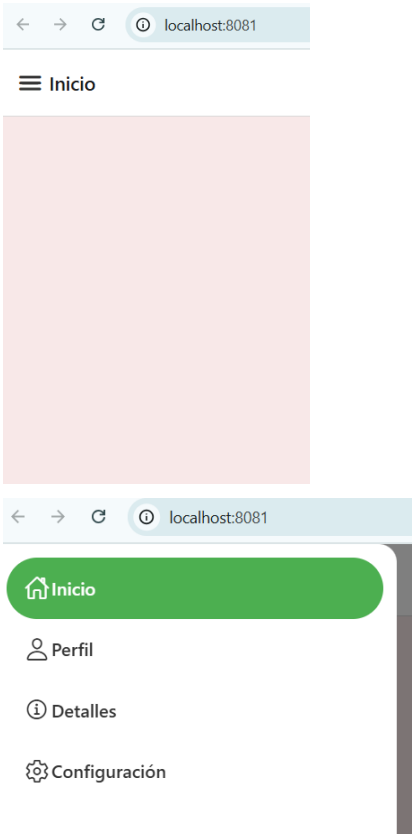
***npm install @react-navigation/drawer***
***npm install @expo/vector-icons***

Crea dentro de la carpeta */navigation* el archivo DrawerNavigator.js con el código de la página siguiente.

Explicación del código:
Se crea un ***menú hamburguesa***, con un icono de la librería ***Ionicons*** de expo vector-icons. Este menú usa el tipo de navegación drawer (menú lateral o cajón) que se desliza (toggle) cuando el usuario pasa el dedo de izquierda a derecha en un móvil.

- Se utiliza el método ***navigation.toggleDrawer()*** que abre o cierra el menú lateral. Otros métodos son: openDrawer y closeDrawer.
- ***screenOptions={({ navigation }) => ({ ... })}*** permite definir opciones globales para todas las pantallas del Drawer.

Resultado:



**DrawerNavigator.js**

navigationApp > navigation > JS DrawerNavigator.js > DrawerNavigator

```jsx
import React from 'react';
import { createDrawerNavigator } from '@react-navigation/drawer';
import { Ionicons } from '@expo/vector-icons';

import ProfileScreen from '../screens/ProfileScreen';
import TabNavigator from './TabNavigator';
import DetailsScreen from '../screens/DetailsScreen';
import SettingsScreen from '../screens/SettingsScreen';


const Drawer = createDrawerNavigator();

export default function DrawerNavigator() {
  return (
    <Drawer.Navigator
      initialRouteName="Tabs"
      screenOptions={({ navigation }) => ({
        headerShown: true,
        headerLeft: () => (
          <Ionicons
            name="menu"
            size={28}
            color="#333"
            style={{ marginLeft: 15 }}
            onPress={() => navigation.toggleDrawer()}
          />
        ),
        drawerActiveBackgroundColor: '#4CAF50',
        drawerActiveTintColor: '#fff',
        drawerInactiveTintColor: '#333',
        drawerLabelStyle: { marginLeft: -10, fontSize: 16 },
      })}
    >
      <Drawer.Screen
        name="Tabs"
        component={TabNavigator}
        options={{
          drawerLabel: 'Inicio',
          title: 'Inicio',
          drawerIcon: ({ color }) => (
            <Ionicons name="home-outline" size={22} color={color} />
          ),
        }}
      />
      <Drawer.Screen
        name="Profile"
        component={ProfileScreen}
        options={{
          title: 'Perfil',
          drawerIcon: ({ color }) => (
            <Ionicons name="person-outline" size={22} color={color} />
          ),
        }}
      />

      <Drawer.Screen
        name="Details"
        component={DetailsScreen}
        options={{
          title: 'Detalles',
          drawerIcon: ({ color }) => (
            <Ionicons name="information-circle-outline" size={22} color={color} />
          ),
        }}
      />
```

```
66
67          <Drawer.Screen
68            name="Settings"
69            component={SettingsScreen}
70            options={{
71              title: 'Configuración',
72              drawerIcon: ({ color }) => (
73                <Ionicons name="settings-outline" size={22} color={color} />
74              ),
75            }}
76          />
77        </Drawer.Navigator>
78      );
79    }
```

color se inyecta automáticamente por React Navigation.
Proviene de las opciones globales del Drawer, en este código de las propiedades que definen cómo se ven los ítems activos e inactivos:

```
screenOptions={({ navigation }) => ({
        headerShown: true,
        headerLeft: () => (
          <Ionicons
            name="menu"
            size={28}
            color="#333"
            style={{ marginLeft: 15 }}
            onPress={() => navigation.toggleDrawer()}
          />
        ),
      drawerActiveBackgroundColor: '#4CAF50',
        drawerActiveTintColor: '#fff',
        drawerInactiveTintColor: '#333',
        drawerLabelStyle: { marginLeft: -10, fontSize: 16 },
      })}
```

Cuando un ítem del Drawer está:

activo: ***color*** será igual a ***drawerActiveTintColor***
inactivo: ***color*** será igual a ***drawerInactiveTintColor***